ADS2002

SOLAR FARM PROJECT

Prepared by Group SF1

Jared Roff
Fatimah Ayub
Regine Chong
Abbey Harford
Tanay Khairnar
Dimitri Argyris

Contents

Executive Summary **2**
Introduction **4**
Data Quality and EDA **5**
Data Cleaning & Data Quality 5
Exploratory Data Analysis 7
Model Development and Results **9**
Weather Forecasting 9
Building & Solar Modelling (LightGBM) 11
Building & Solar Modelling (ARIMA) 16
Scheduling Optimisation (Gurobi) **18**
Model Version 1 18
Model Version 2 - Final Version 19
Conclusion **22**
Acknowledgments **23**
References **23**
Appendix **24**

Executive Summary

Problem

This project addressed the IEEE Computational Intelligence Society's Predict and Optimise Challenge, which required the development of a framework to forecast and optimise renewable energy scheduling for November 2020. This task focused on predicting the electricity demand and solar power production from six buildings at Monash University, and subsequently using these forecasts to optimise the timetable and solar battery usage. The predominant challenge was integrating accurate forecasting with effective optimisation to reduce energy costs.

The Data

The datasets provided historical time series up to September 2020, consisting of:

- Electricity demand from six buildings at Monash University, and solar power generation from six solar panels, recorded at 15-minute intervals.
- Weather data including daily minimum and maximum temperatures, rainfall and solar exposure of three weather stations across Melbourne.
- Electricity price data provided from the Australian Energy Market Operator.
- Problem Instance files which were split into two phases, with phase one being up until September 30th, 2020, and phase two being October 2020. These files described the configuration of the buildings, solar panels, batteries and scheduled activities. These files were crucial for optimisation, as they defined the resources available and the precedence constraints that had to be satisfied.

Process

The process for optimising the timetable was as follows:

1. Weather Forecasting: We forecasted the maximum and minimum temperatures, rainfall and solar exposure of October 2020 using Light Gradient-Boosting Machine (LightGBM) modelling. We then compared these results to the actual values for October, refined the model and then attempted to forecast November 2020.
2. Solar & Building Forecasting: The weather forecasts were then used as inputs for modelling electricity demand and solar generation across the same time periods. Two different models were attempted, specifically Autoregressive Integrated Moving Average (ARIMA) and LightGBM.
3. Using the weather-driven forecasts, and factoring in constraints, and penalties based on when classes are scheduled, the activity timetables and battery schedules were optimised to optimise costs.

Findings

The project resulted in three main outcomes:

1. Weather Forecasting: Whilst the in-sample forecast for October had high accuracy, November was susceptible to inaccuracy due to attempting to predict multiple target variables without the availability of future weather variables available during the training of the October forecast.
2. Solar & Building Forecasting: It was found that using LightGBM for modelling performed better for solar production, producing a Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) of 2.43 and 4.02 compared to ARIMA's 3.89 and 4.55. However, ARIMA was more accurate for building demand as LightGBM was defined as a global model with little parameter tuning done, and attempting to generalise the different buildings resulted in higher error measures. ARIMA resulted in a Mean Absolute Percentage Error (MAPE) of 13.86% compared to LightGBM with a MAPE of 59.07%. Despite ARIMA's better performance for buildings, consistent memory errors meant we could not pursue the model further.
3. Timetable Optimisation: The final optimised timetable saved $823,498, and was 13.49% more efficient than the instance file schedule, whilst its peak grid draw was 49.6% of the peak grid draw from the instance files.

Limitations and Recommendations

A key limitation of this project was the difficulty of accurately forecasting weather for a month in advance without access to all the required feature variables. The limited number of weather stations and data being collected at the relevant weather stations reduced the predictive strength of the weather forecasting. Additionally, the use of a global LightGBM model without fine-tuning for individual buildings led to higher error measures for the building electricity demand forecasts. The ARIMA model may have also been a suitable choice for predicting building demand if there were no memory errors.

To improve the forecasting accuracy, future work could explore incorporating further external weather datasets. Additionally individual models for each building could be beneficial in predicting the electricity demand. Furthermore, the use of neural networks could enhance future forecasting performance.

Conclusion

This project demonstrated the potential for data-driven optimisation to enhance energy efficiency for university systems. Through the use of combining predictive modelling with constrained optimisation, the framework provides a scalable foundation for renewable energy scheduling in real-world applications.

Introduction

What is the IEEE-CIS Predict and Optimise Challenge?

The IEEE Computational Intelligence Society's Predict and Optimise Challenge was a competition designed to test the combined use of forecasting and optimisation related to renewable energy. There were two tasks, first to forecast electricity demand and solar generation from Monash University's campus, and second, to use those forecasts to optimise the building activity timetables and solar battery usage. The primary objective is to minimise overall energy costs , while still satisfying the timetabling requirements and constraints.

This challenge reflects issues faced in the real-world in modern energy systems. Renewable energy sources such as solar power are intermittent and weather-dependent, which makes accurate forecasting a critical component of energy management. As noted in a study "accurate forecasting of solar power generation is essential for optimising renewable energy utilisation" (Wan et al., 2025). This directly relates to the focus of this project as the forecasts of temperature, rainfall, and solar exposure were utilised to predict electricity demand and solar output. From there, those forecasts then formed the foundation for the optimisation process, ensuring that the timetables and battery usage were scheduled in a way that reduced costs.

Our Data

The datasets provided for this project combined different streams of information, including:

Forecasting Data

Building demand and solar production: The data was provided in a Time Series Format (.tsf) file containing electricity consumption and solar generation data measured in kWh from six Monash University buildings, each equipped with rooftop solar panels. Data were recorded at 15-minute intervals, with different start dates ranging from 2016 to 2019. Most solar series began around 2019. Phase 1 included data up to 30 September 2020, while Phase 2 extended this to 31 October 2020, enabling an additional month for model evaluation.

Weather data: Daily minimum and maximum temperatures, rainfall, and solar exposure recorded across three Melbourne weather stations, sourced from the Australian Bureau of Meteorology (BOM). These variables were used to forecast future weather data as well as support the forecasting of building demand and solar generation.

Optimisation Data

Problem Instance files: Described the configuration of each building, solar system, and battery, as well as the scheduled activities and precedence constraints that defined the optimisation setup. These were also split into Phase 1 and Phase 2 files.

Electricity price data: Provided by the Australian Energy Market Operator (AEMO), used together with the instance files to determine the optimal timetable for energy use and storage based on the forecasted demand and generation.

The datasets included values up until September 2020, with the competition requiring forecasts for November 2020. This project utilised weather data up to November 2020, with the month of November's observations being employed as 'out of sample' data for model/forecast evaluation. Combined, these datasets provided the foundation for the solar farm project.

Data Quality and EDA

Exploratory Data Analysis (EDA) was a foundational step in this project, serving to not only familiarise ourselves with the dataset but also uncover key insights, identify seasonal trends and outliers, and prepare the data for forecasting and optimisation.

Data Cleaning & Data Quality

Phase 1 Data Cleaning

To prepare the Phase 1 dataset for analysis, the raw .tsf file was converted into a consolidated data frame. Although the provided utility file could perform this conversion, it caused issues when generating plots during exploratory data analysis (EDA). To address this, a custom parsing function was developed to read the .tsf file line by line, extract series names, timestamps, and values, and construct a unified data frame. The function also included a relabeling step to ensure consistency, aligning building and solar series numerically as Building 1–6 and Solar 1–6. Since the original building numbering was offset, series originally labeled 4–7 were renamed to 3–6 for alignment. The same procedure was applied to the Phase 2 dataset to maintain consistency, but only the October 2020 segment was extracted and used to evaluate model performance.

The consolidated data frame was then split into 12 smaller data frames - one for each building and solar series - to facilitate individual-level EDA. As timestamps were recorded in Coordinated Universal Time (UTC), they were converted to Australian Eastern Standard Time (AEST) to reflect the local context of Melbourne, ensuring all analyses were temporally consistent.

Next, we assessed the missingness across the 12 series, producing a summary data frame of total and percentage missing values (**Figure 1**). Results showed substantial gaps in some building series, while solar data contained long zero sequences indicating data quality issues. Buildings 2 and 3, with little missingness, were excluded. The remaining buildings and all solar series underwent imputation to restore continuity and prepare the dataset for analysis and modelling.

	name	num_missing	missing_percent
0	Building1	47404	32%
1	Building2	88	0%
2	Building3	561	0%
3	Building4	17661	40%
4	Building5	29992	72%
5	Building6	2255	5%
6	Solar1	0	0%
7	Solar2	0	0%
8	Solar3	0	0%
9	Solar4	0	0%
10	Solar5	0	0%
11	Solar6	0	0%

Figure 1 - data frame showing completeness on each building and solar panel

Weather Data Cleaning

The weather data was downloaded from the Bureau of Meteorology website as individual .CSV files for each type of observation (ie. rainfall, solar exposure, temperature) at weather stations including Melbourne Olympic Park, Moorabin and Oakleigh. This constituted 10 files that were to be combined into one clean dataset for manipulation and modelling. Though the Solar Farm Competition provided TSF files that aimed to automate this process, the BOM website did not permit the web scraping technique utilised, resulting in corrupted file downloads. Instead, files were manually concatenated and organised by a common ‘Date’ variable. Early in the project, the concatenation method was identified as flawed, as each weather observation of each day was assigned an individual row, accumulating many false NaN values within the weather dataset and corrupting analysis. The final ‘weather_data_fixed.csv’ merged all files on the common ‘Date’ variable.

Building Imputation

To address the missing values identified during data cleaning, different imputation strategies were applied depending on the percentage of missing values in each building series. For Buildings 1, 4, and 6, where the proportion of missing data was low to moderate, a seasonal average imputation method was used. This approach used the strong seasonal patterns in the data by grouping values according to month, day of the week and hour of the day. Missing entries were then filled in with the corresponding group mean (Junninen et al., 2004). This ensured that imputed values reflected the expected seasonal and daily demand behaviour. Median imputation was also trialled but produced results that were nearly identical to the mean-based approach.

In contrast, Building 5 had over 70% of its data missing, making seasonal averages unsuitable. To handle this, a k-Nearest Neighbours (kNN) imputation method was applied. This technique used the values of similar time points (based on features such as hour, day of the week, and month) to infer plausible replacements for the missing entries. The kNN approach allowed for a better recovery of the time series structure in cases of extensive missingness as seen in **Figure 2.1** and **2.2**, ensuring that Building 5 could still be incorporated into subsequent analyses. Similar to the spatio-temporal methods proposed by Kuppannagari et al. (2021) for smart power grid data, this approach utilised temporal dependencies to reconstruct missing segments and maintain realistic demand behaviour.

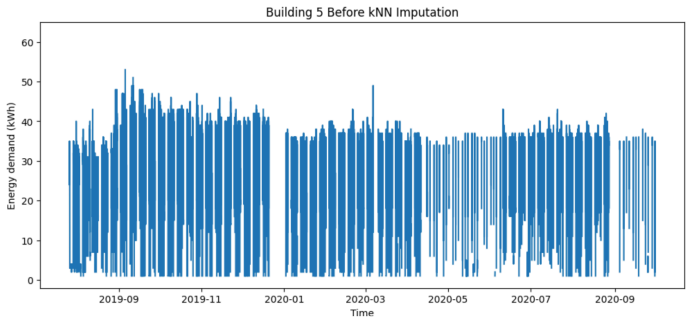

Figure 2.1 - Building 5 Before kNN Imputation

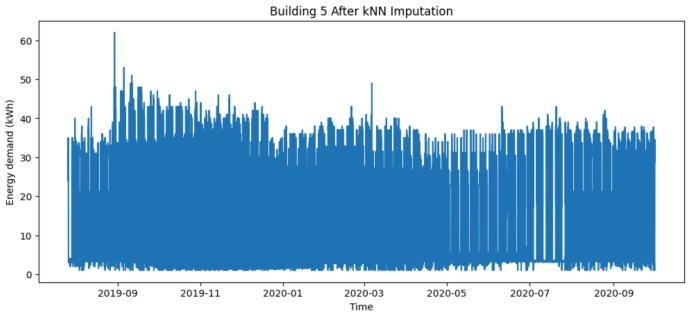

Figure 2.2 - Building 5 After kNN Imputation

Solar Imputation

The imputation process for the solar data required a different strategy compared to the building dataset. The main challenge was not the presence of missing values but the abundance of zero readings, which rendered conventional imputation techniques such as mean or median substitution ineffective. Upon reviewing reports from the actual competitions, we found that some opted to remove all zero values. However, this approach was unsuitable because models such as LightGBM require complete datasets. On the other hand, algorithms such as XGBoost were capable of handling missing values, providing more flexibility in data preparation. Our initial attempt using forward and backward filling proved to be unreliable due to the scarcity of valid non-zero values, which led to inaccurate temporal patterns. Ultimately, as a temporary solution, we removed zero values recorded during daylight hours specifically from 6 AM to 10 PM. This adjustment was based on the physical expectation of solar activity and helped improve data realism, ensuring that the imputed dataset better reflected true solar generation behaviour.

Exploratory Data Analysis

To begin the exploratory data analysis (EDA) for the Phase 1 dataset, we first examined the descriptive statistics for each building and solar panel. The dataset contained energy demand and solar production data for six buildings and six solar panels, recorded at 15-minute intervals. To summarise these patterns, we grouped the data by building and calculated key statistics including the mean, standard deviation, minimum, and maximum values. As shown in **Figure 3**, Buildings 1 and 3 exhibited the highest average demand and the greatest variation, whereas Building 4 consistently showed the lowest values with minimal variability. In comparison, all solar panels had considerably lower production levels than the buildings' energy demands, with the exception of Building 4, which aligned more closely with its corresponding solar panel. This early observation highlighted a potential challenge for the optimisation process - building demand generally exceeded solar generation capacity.

	series	mean	std	min	max
0	Building1	230.11	133.54	0.1	6781.50
1	Building2	11.37	7.91	1.5	83.50
2	Building3	517.94	273.81	85.0	8556.00
3	Building4	1.33	0.53	1.0	5.00
4	Building5	24.67	11.10	1.0	62.00
5	Building6	30.71	5.59	0.2	102.20
6	Solar1	4.39	8.86	0.0	50.41
7	Solar2	1.94	3.43	0.0	13.04
8	Solar3	1.88	3.12	0.0	13.96
9	Solar4	1.36	2.21	0.0	11.04
10	Solar5	1.12	1.89	0.0	8.10
11	Solar6	3.89	8.75	0.0	40.43

Figure 3 - Descriptive statistics of each Building and Solar

To gain a clearer visual understanding of these relationships, we plotted the total building demand and total solar production across all six sites on a single graph. The data were aggregated at 15-minute intervals to reflect the original granularity of the dataset. However, to better identify temporal trends, we also developed visualisations at hourly, daily, weekly, and monthly resolutions. This flexibility allowed us to explore how energy usage and generation varied across different time scales. For instance, the daily aggregation (**Figure 4**) revealed that overall building demand was consistently higher than solar production, reinforcing the pattern observed in the descriptive statistics. These visualisations also provided a useful reference for the forecasting stage, helping us anticipate the approximate range and seasonal trends of both demand and production values.

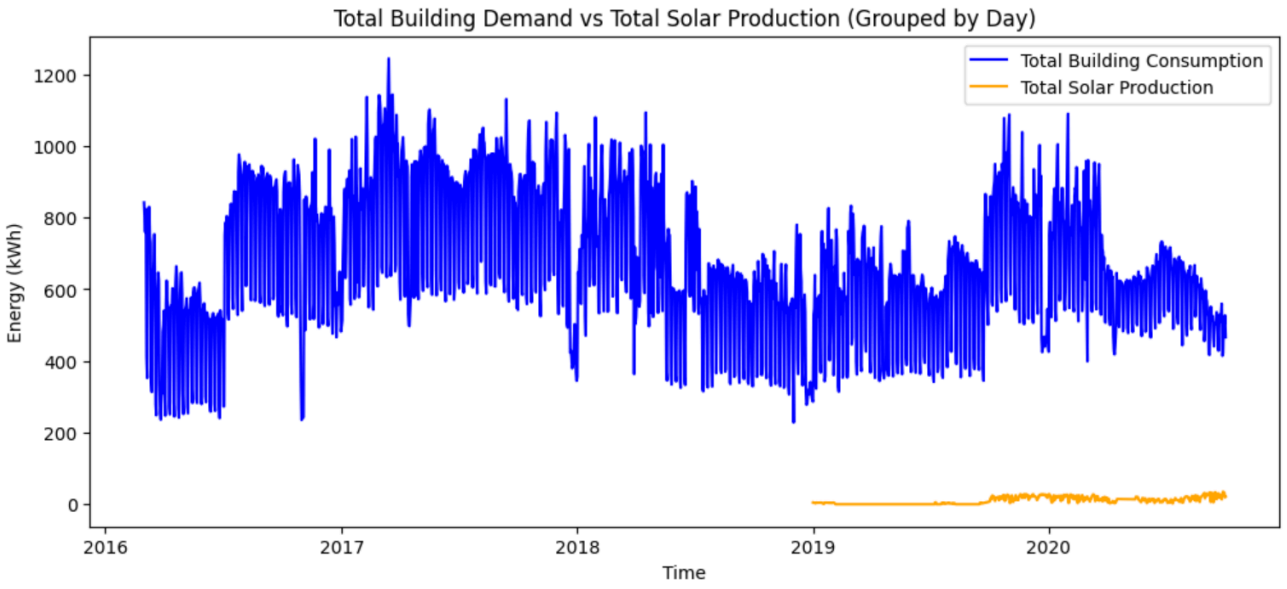

Figure 4 - Plot showing total building demand and total solar production

To further investigate temporal patterns, we classified each data point into one of four time periods: Morning (6 a.m.–12 p.m.), Afternoon (12 p.m.–4 p.m.), Evening (4 p.m.–10 p.m.), and Night (10 p.m.–6 a.m.). This classification, implemented via a custom function (see Appendix A), allowed us to calculate mean demand and production for each time period across all buildings and solar panels. In general, buildings showed higher energy demand than solar production throughout the day, reinforcing the patterns observed in the earlier descriptive statistics and total demand vs. production plots. Both demand and solar generation were typically highest in the afternoon, while buildings maintained relatively high night-time demand and solar panels had no production during these hours. These observations highlight the daily imbalance between consumption and generation and indicate when reliance on stored or external energy sources would be most critical.

In addition, a correlation analysis of the six solar panels showed very high correlations, mostly above 0.9 (**Figure 5**). This indicates that solar generation across all sites is strongly influenced by common factors, such as sunlight and weather conditions, reinforcing the importance of including weather features in the forecasting models to accurately capture variations in solar output.

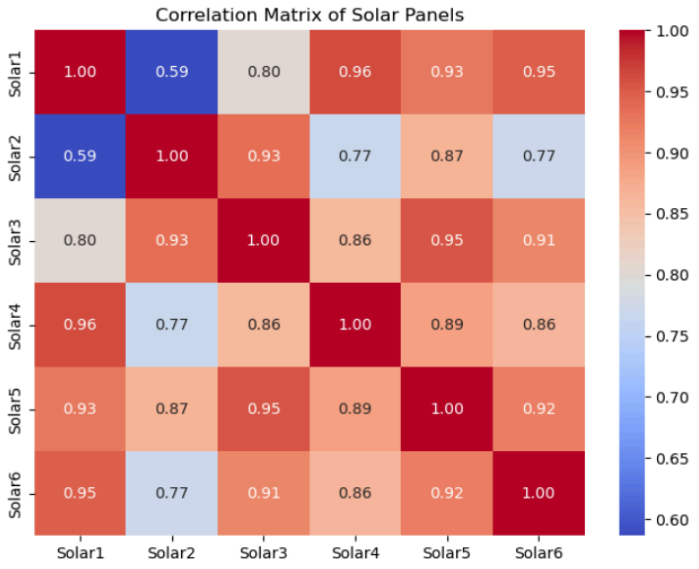

Figure 5 - Correlation heatmap of Solar Panels

Weather Data EDA

From the cleaned weather dataset, we examined yearly seasonal patterns, which were expected given Melbourne's climate. Analysis of the data showed clear cyclical trends, with winter months (June–August) averaging around 10°C and summer months reaching close to 20°C (**Figure 6**). These patterns were consistent across multiple years, confirming the reliability of the dataset and providing a useful context for understanding how temperature variations might influence building energy demand and solar generation.

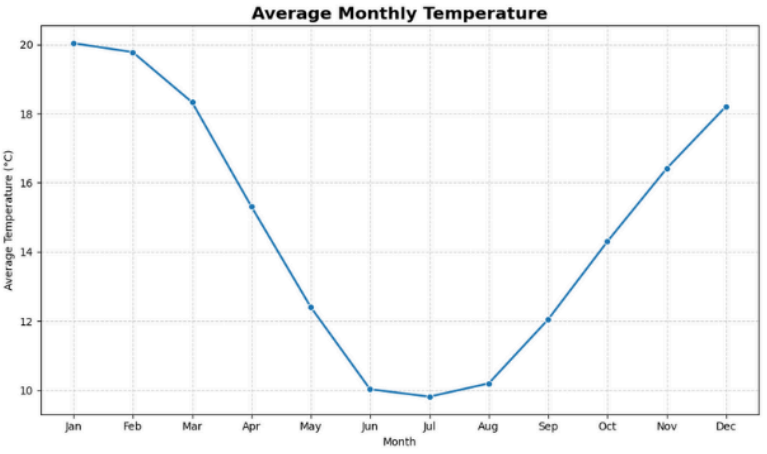

Figure 6 - Line plot of average monthly temperature from cleaned weather data

Based on the identification of these cyclical patterns, the weather dataset was altered to create 'sin_doy' and 'cos_doy' feature variables that converted the existing 'Day of Year' variable (ranging from 1 to 365) into spherical coordinates that could assist the forecasting models to learn seasonal weather patterns within the data.

Model Development and Results

Weather Forecasting

XGBoost: Extreme Gradient Boosting (XGBoost) is a machine learning algorithm based on gradient boosting decision trees, growing trees by level so that all leaves are expanded to the same depth before increasing the depth of the tree. This creates more balanced trees that are less prone to overfitting than other decision tree based models. An advantage of XGBoost is its automatic handling of missing values, allowing full utilisation of the weather dataset despite the presence of NaN values. Initial models for in-sample forecasting were implemented using XGBoost to gain a basic understanding of possible retrospective weather forecasting model accuracy. The weather forecast model for October is illustrated in **Figure 7**. We can see that the model achieves a very high R^2 score of 0.9928, and MAE of 0.2687. Whilst these results are strong, it does not provide insight to the accuracy of how the model will perform on out of sample data.

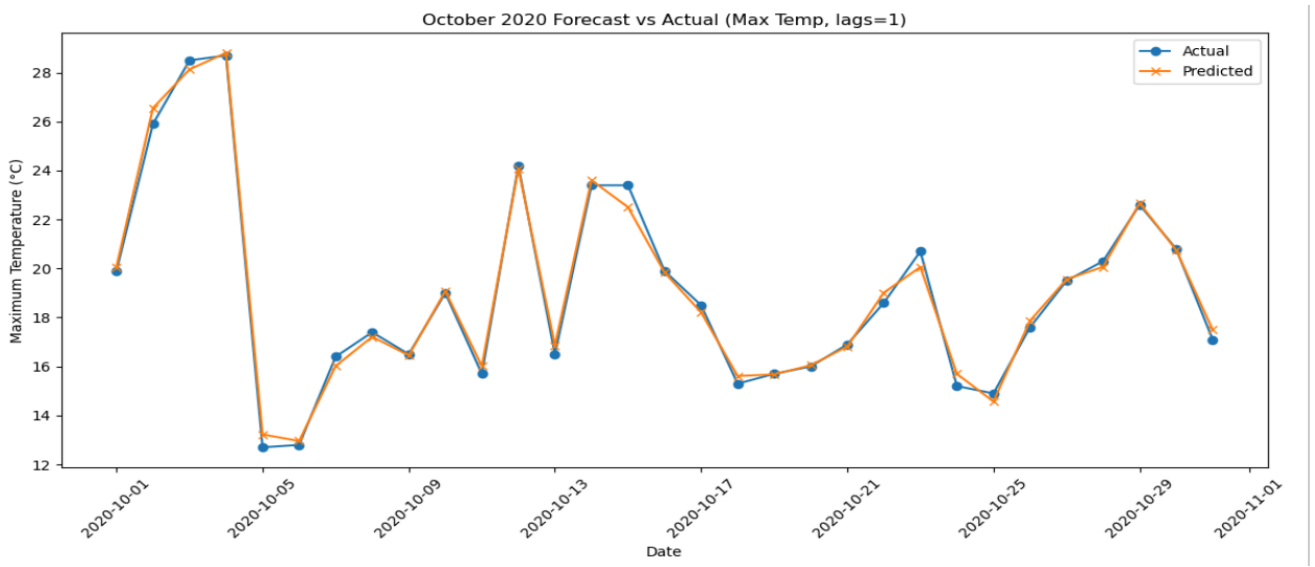

Figure 7 - October 2020 Weather forecast using XGBoost

LightGBM: Light Gradient Boosting Machine is an advanced machine learning algorithm based on gradient boosting decision trees. It builds an ensemble of decision trees iteratively, where each new tree corrects the errors of the previous ones. It uses leaf-wise growth, which allows it to achieve higher accuracy than traditional level-wise methods as it splits leaves with the largest loss reduction. This makes the LightGBM models better at capturing non-linear data and local extremes than XGBoost.

Both XGBoost and LightGBM were implemented in conjunction with Scikit Learns MultiOutputRegressor function that allowed multi-target regression where models predicted more than one target variable at a time. MultiOutputRegressor predicts the targets independently, and whilst there exists a function to predict multiple targets that are correlated (RegressorChain from Scikit Learn), preliminary model metric evaluations exhibited in **Table 8** found RegressorChain to have marginally lower R^2 scores and slightly higher MAE than MultiOutputRegressor outputs for the XGBoost model, and thus the latter was utilised for the final model.

Model Composition	MAE	R2
XGBoost RegressorChain	0.879	0.924
XGBoost MultioutputRegressor	0.825	0.928
LightGBM RegressorChain	0.757	0.936
LightGBM MultioutputRegressor	0.757	0.936

Table 8 - Table comparing MAE and R^2 of different XGBoost and LightGBM models

Additionally, these preliminary tests also identified the superior decision tree based model for the weather forecasting to be LightGBM, as it produced a higher R^2 score of 0.936 compared to XGBoosts 0.928, as well as a smaller MAE of 0.757 compared to 0.879.

To reconstruct the testing data frame without data leakage, 7 days of lags and 3/7 day rolling averages were created for each target variable, along with exogenous feature variables including spherical coordinates, month, day_of_year. To examine the feature importance of these variables, a SHAP plot was created, evident in **Figure 9.**

This SHAP plot highlights the features that have the most influence on the models predictions. We can see that the maximum temperature of the previous day is treated as the most important predictive feature for the weather forecasting model. The SHAP library functions (implementing Shapley Additive Explanations to explain the outputs of machine learning models) were also employed to prune any superfluous variables that contributed negligibly to the model, but all variables were deemed above the importance threshold of 0.01.

The Optuna library was used to optimise the parameters of the model, resulting in a model with a learning rate around 0.012, 19 leaves, 270 estimators, Lasso and Ridge regression coefficients of 1.2649 and 3.0577 respectively based on 30 trials.

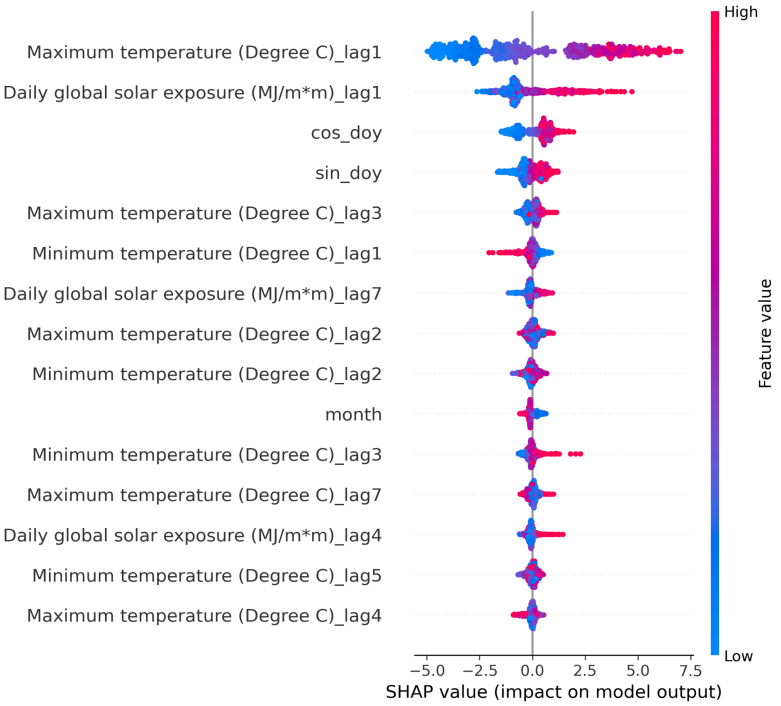

Figure 9 - SHAP Feature Importance Plot for LightGBM Weather Forecasting Model

A final optimisation of the weather forecast was adding an additional target, ‘daily_average’ to our target variable set, which resulted in lower error metrics than an identical model without the additional target.

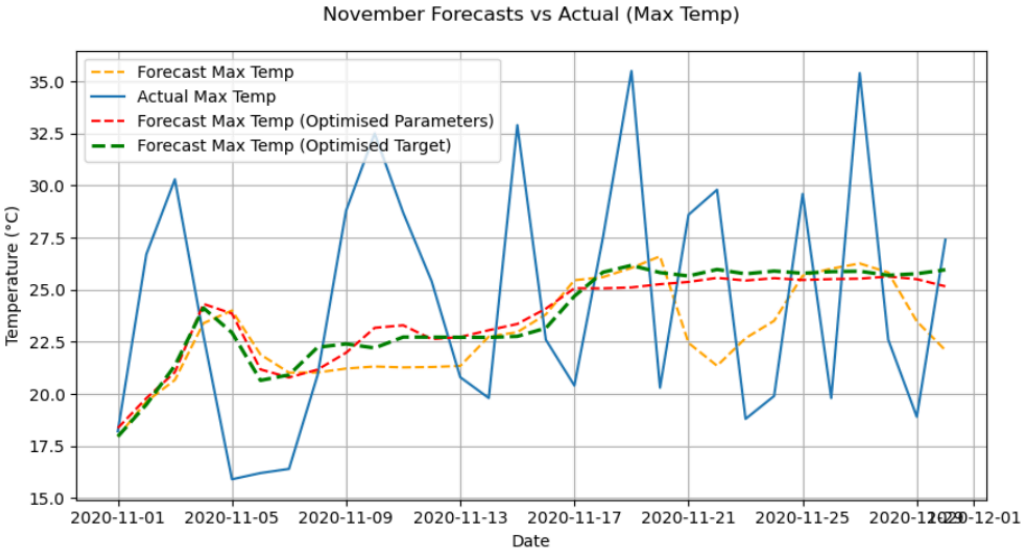

Figure 10 - Optimisation Iterations of Final LightGBM Weather Forecast for November 2020

Figure 10 shows the results of the optimisation iterations, with yellow modelling initial LightGBM model, pink showing Optuna optimised parameters and finally the green modelling the implementation of 'daily_average' to the target set. The final models metric evaluations are exhibited in **Figure 11**. The optimised target (green) model has the lowest RMSE and MAE, as well as the only positive R^2 score of 0.09. The high errors and limited capacity to capture the variance in the actual November weather can be attributed to the absence of rolling features like 'Days of Accumulation of Maximum Temperature' which help the model to make stronger predictions and not just predict close to the average. The 'Optimised Target' model is the final weather prediction model for November 2020, and had an average error of 4.9 degrees.

```
=== Forecast Metrics ===
Original Forecast:
{'MSE': 37.307516248839285, 'RMSE': 6.107987905099296, 'MAE': 5.274001770342227, 'R2': -0.13445593656554977}

Optimised Parameters Forecast:
{'MSE': 33.03275758603568, 'RMSE': 5.747413121225555, 'MAE': 4.992780270441469, 'R2': -0.004468045920231312}

Optimised Target Forecast:
{'MSE': 32.57913991582415, 'RMSE': 5.707813934933772, 'MAE': 4.908533075628349, 'R2': 0.009325669412359683}
```

Figure 11 - Metrics for Optimised November Forecast Models

Building & Solar Modelling (LightGBM)

Building Forecasting Training Model

LightGBM was also used to predict building demand using both time-series and weather-related features.

The 15-minute building data was merged with the daily weather data based on date alignment so that each record included corresponding weather conditions. After merging, new features were then added to capture time patterns such as hour, day of week, month, and whether it was a weekend. Lag features were created to represent past values

(15-minute, 1-day, 7-day lags), and rolling averages and standard deviations were used to capture short-term and long-term trends. Rows with missing lag or rolling values were dropped to keep the data clean.

Two modelling strategies were used: a global model and separate per-building models. The global model trained on all buildings together, using the building name as a categorical feature. It used 1500 estimators, a learning rate of 0.03, and parameters like subsample and colsample_bytree of 0.8 to control overfitting. Data was split by time, with the last 30 days used for validation. Performance was measured using RMSE, MAE, and R^2 scores (**Figure 12)**.

```
[Global model]  RMSE=4.385  MAE=1.546  R2=0.9991
```

Figure 12 - Performance of global LightGBM model on Building training data

The per-building models were trained individually for each building, using fewer estimators and a slightly higher learning rate for faster training. These models could better capture each building's unique energy pattern. Evaluation showed that the global model worked well overall, per-building models performed slightly better in some cases (**Figure 13)**.

```
Per-building models:
  Building1: RMSE=14.549, MAE=6.554, R2=0.900
  Building2: RMSE=0.324, MAE=0.165, R2=0.996
  Building3: RMSE=8.590, MAE=2.674, R2=0.991
  Building4: RMSE=0.465, MAE=0.278, R2=0.680
  Building6: RMSE=0.675, MAE=0.338, R2=0.986
```

```
Per-building (global model):
  Building1: RMSE=6.230, MAE=3.183, R2=0.982
  Building2: RMSE=0.576, MAE=0.324, R2=0.988
  Building3: RMSE=6.175, MAE=2.338, R2=0.995
  Building4: RMSE=0.537, MAE=0.386, R2=0.573
  Building6: RMSE=0.870, MAE=0.429, R2=0.977
```

Figure 13 - Comparison of global model to per-building model

Feature importance results showed that lag and rolling features were the most useful (**Figure 14)**, meaning past energy usage strongly influenced future demand. Weather variables such as temperature and solar exposure also contributed to prediction accuracy.

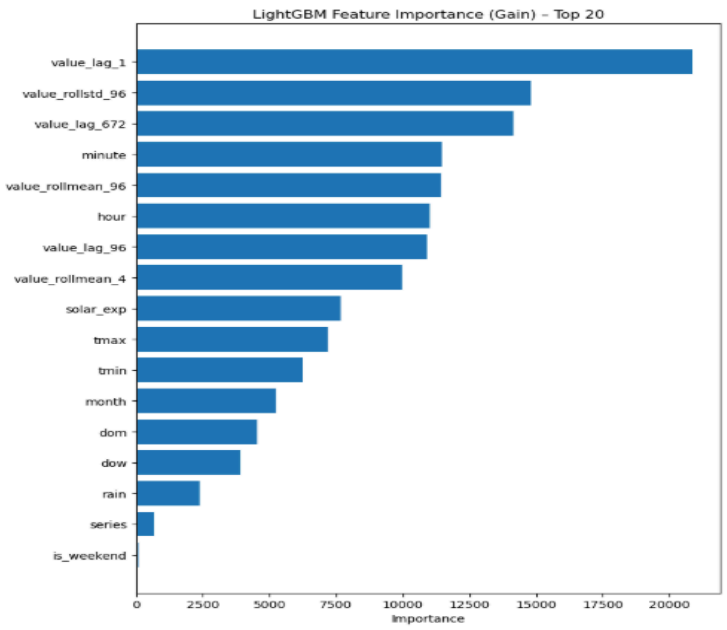

Figure 14 - Horizontal bar chart showing the key features for Building prediction

Solar Forecasting Training Model

LightGBM is also used to forecast solar production from a 15-minute series enriched with daily weather. The data pipeline loads cleaned solar readings and daily weather. Weather dates are parsed, core columns are renamed to tmax, tmin, rain, and solar_exp, and gaps are filled with forward/backward fill. Solar records are sorted by series and timestamp, a calendar date is derived, and daily weather is merged on that date so each 15-minute point carries its day's weather.

Time features capture routine seasonality: minute, hour, day-of-week, day-of-month, month, and a weekend flag. To inject history, lag features reference prior values at 15 minutes, 1 day (96 steps), and 7 days (672 steps). Rolling windows provide short/long-term structure with 1-hour and 1-day means plus a 1-day standard deviation, all computed with a one-step shift to avoid leakage. Rows with NaNs introduced by these windows are dropped.

```
[Global model]  RMSE=0.613  MAE=0.224  R2=0.9960
```

Figure 15 - Performance of global LightGBM model on Solar training data

Training uses a strict time-based split, where all the data before the last 30 days was used for training, and the most recent 30 days were reserved for validation. Certain columns such as the timestamp, date, and time period were excluded from training, while the series label was treated as a categorical feature. The LightGBM model was trained with a moderate learning rate and a balanced number of decision tree leaves to ensure both accuracy and generalisation. It also used subsampling techniques to reduce overfitting and improve stability. The model's performance on the validation set was then evaluated using RMSE, MAE, and R^2 (**Figure 15**).

Feature importance results show that the model's predictions closely follow weather patterns (**Figure 16**). The most influential variables include lag and rolling features, which capture how recent production trends affect future output, and weather-related factors such as solar exposure, maximum temperature, and minimum temperature. These weather variables play a key role in determining solar generation, confirming that the model successfully learns the relationship between sunlight and temperature conditions and the amount of solar energy produced.

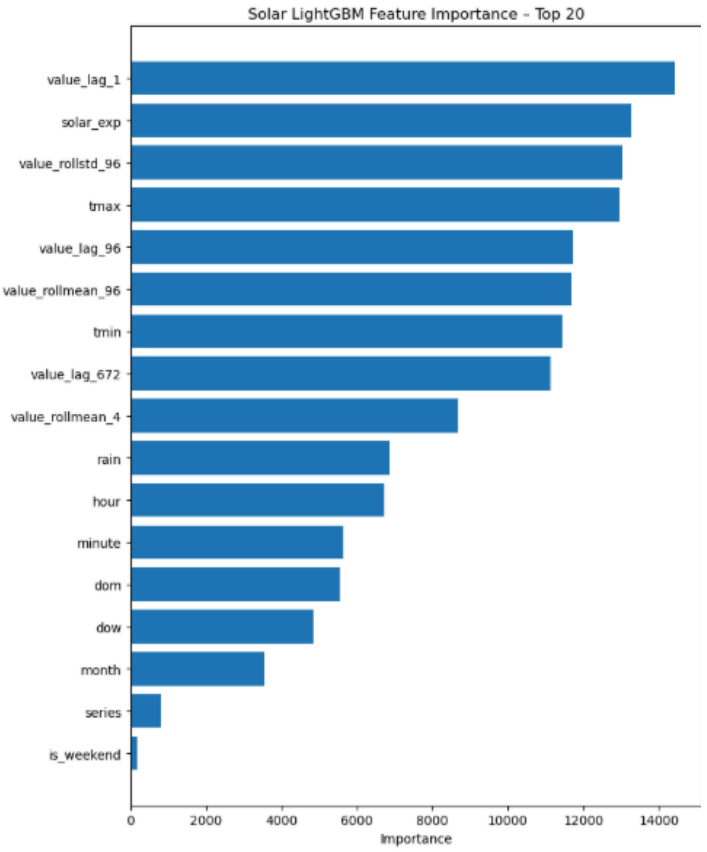

Figure 16 - Horizontal bar chart showing the key features for Solar prediction

Forecasting Using LightGBM in R

The LightGBM model was implemented in R to forecast both solar and building energy outputs, extending the previous work conducted in Python. To maintain consistency, the model utilised the same lag variables, rolling means, and weather-based features as in the Python version, along with identical hyperparameters such as learning rate and number of leaves to ensure a fair comparison. Once this structure was established, switching between different datasets, such as from one building to another or between solar panels by changing the series name to the corresponding dataset. Despite using the same features and parameters, the results produced in R differed slightly from those generated in Python. Nevertheless, the R implementation proved effective, as the plots demonstrated strong predictive performance for October 2020 when compared with the actual Phase 2 dataset.

Forecasting Using LightGBM in R (Buildings)

Figure 17 above compares the predicted and actual energy demand for Building 2, showing that while the LightGBM model successfully captures the general consumption trend, it struggles to detect short-term fluctuations, leading to occasional underprediction. The model effectively learns seasonal and daily variations in building demand, aided by the inclusion of the weather dataset as an exogenous feature. However, the predicted values exhibit slightly higher peaks than the actual data, suggesting that the model gives more weight to high historical averages, which inflates the predicted magnitudes.

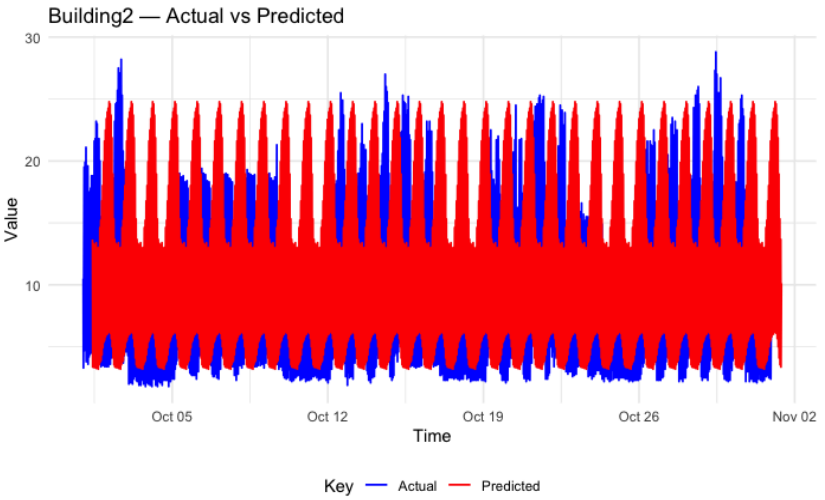

Figure 17 - Comparison of Building 2 October predictions to actual October values

The LightGBM model achieved a MAE of 4.81 and a RMSE of 5.96, indicating that the forecasts were generally close to the observed values, with only minor deviations. Using this model, we predicted for November 2020 (**Figure 18)**. Overall, LightGBM demonstrates strong predictive capability for the building dataset, though its performance could be enhanced by incorporating additional explanatory features. Furthermore, building-specific feature sets could be used rather than applying the same feature configuration across all buildings, allowing the model to better capture the unique consumption patterns of each site.

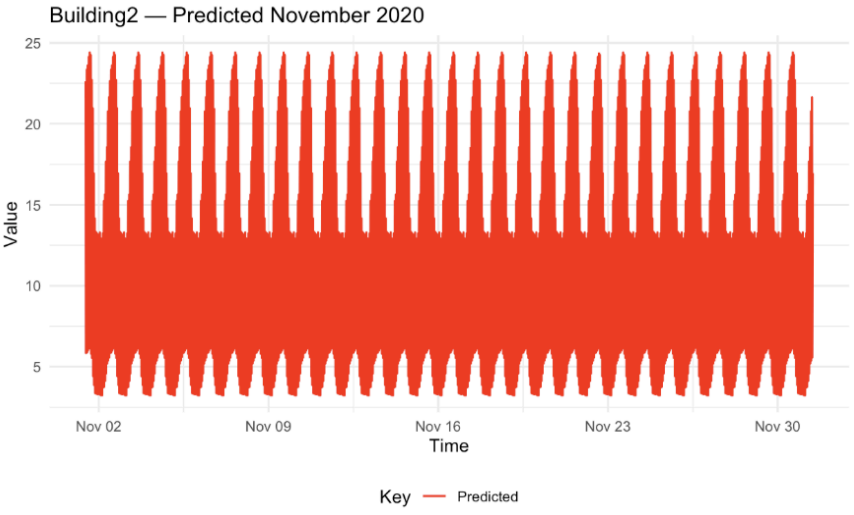

Figure 18 - Prediction of Building 2 for November 2020

Forecasting Using LightGBM in R (Solar)

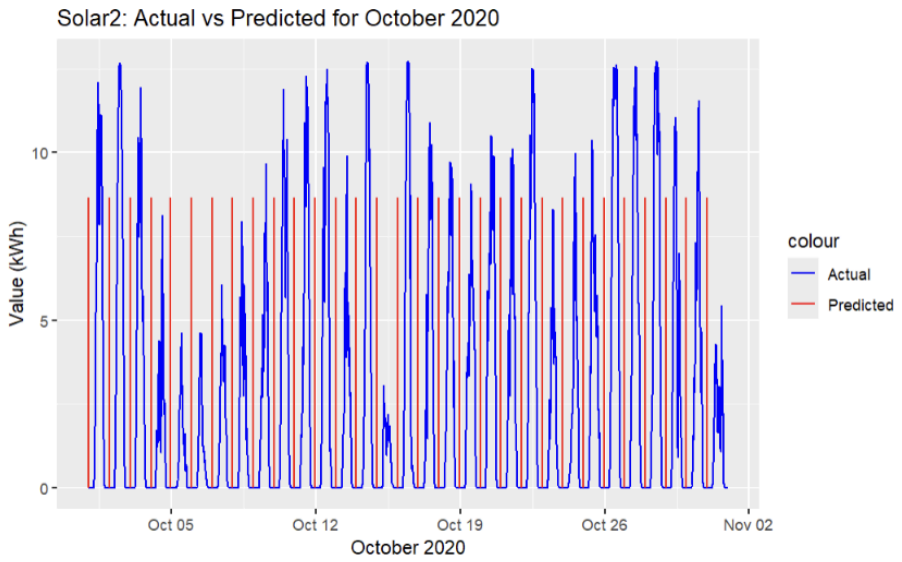

Figure 19 - Comparison of Solar 2 October predictions to actual October values using LightGBM

The comparison between actual and predicted solar energy output for Solar 2 during October 2020 (**Figure 19)** shows that the LightGBM model effectively replicated the daily production cycle of solar generation, capturing the rise and fall in output associated with daylight hours. On the other hand, it consistently underestimated actual generation. This underprediction suggests that the model was somewhat conservative, likely due to limited representation of weather-related features. The model produced MAE of 2.43 and RMSE of 4.02 for the solar energy forecasts, indicating that the model produced predictions that were generally close to the real data. These relatively low error values suggest that LightGBM effectively captured the overall solar generation trend, but these variations occurred due to the availability of sunlight or other weather conditions. The narrower range and smoother appearance of the predicted values indicate that LightGBM generalised the pattern rather than responding to the daily fluctuations seen in the actual data which are generally due to sunlight intensity. Despite these flaws, the model's ability to maintain consistent periodicity implies strong seasonality recognition, making it a valuable forecasting tool for planning solar energy availability. Future improvement could involve incorporating meteorological variables and higher-frequency data to better model short-term variability and improve peak accuracy. From this we can predict November 2020 as shown in **Figure 20**.

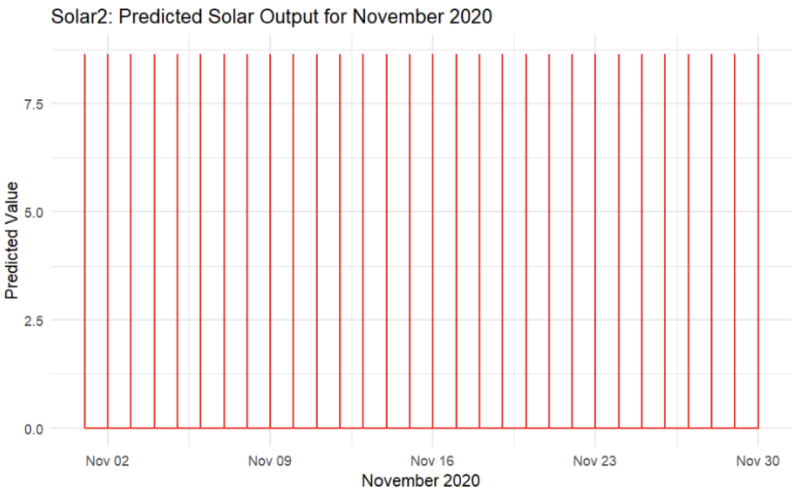

Figure 20 - Prediction of Solar 2 for November 2020 with LightGBM

Building & Solar Modelling (ARIMA)

Building Forecasting

The ARIMA (AutoRegressive Integrated Moving Average) model is widely used for forecasting univariate time series, particularly in energy systems where temporal dependencies are strong (Ugbehe et al., 2025). It models a variable based on:

- Auto-Regressive (AR) terms (p): influence of previous observations.
- Integrated (I) terms (d): differencing applied to achieve stationarity.
- Moving Average (MA) terms (q): impact of past forecast errors.

The SARIMA extension adds seasonal components, while SARIMAX allows exogenous features. Here, the exogenous variables included dayofweek and hour to account for weekday/weekend patterns and daily usage cycles (Kontopoulou et al., 2023).

Due to memory constraints, Building 6's 15-minute data was aggregated to hourly intervals. Auto ARIMA failed to converge on both 15-minute and hourly data, so a manual SARIMAX modeling approach was adopted. Orders were selected via ACF and PACF plots - which help select p and q values - with differencing set to 0 after stationarity checks. Implausibly high demand values (>45) were removed, and the series was split into training (80%) and testing (20%) sets, aligned with exogenous features. The chosen model, SARIMAX(2, 0, 1) × (1, 1, 1, 24), captures both short-term correlations and daily seasonality (24-hour cycle). The exogenous features (dayofweek and hour) allow the model to account for predictable variations associated with the day and time, consistent with prior ARIMA-based forecasting work in renewable and grid stability contexts (Prakash et al., 2025).

Upon fitting the model to the training data, the model's forecasts were compared against the test set (**Figure 21)**. Visual inspection shows that the model captures the general trend of energy demand, but finer fluctuations are not fully captured. On the test set, the model achieved a MAE of 2.8, RMSE of 3.6, and MAPE of 8.8%, indicating reasonable predictive accuracy for typical demand patterns. To evaluate the model's forecasting capability, it was refitted on the full dataset through September 30, 2020, and used to forecast the 744-hour period of October 2020 (**Figure 22)**. When compared with the actual observed demand, the model again captured the general trend but failed to account for more complex fluctuations and sudden changes in demand. Forecast metrics for October were MAE of 3.9, RMSE of 5.1, and MAPE of 13.86%, showing a decline in predictive accuracy as the model struggled with non-stationary and highly variable patterns in the unseen data.

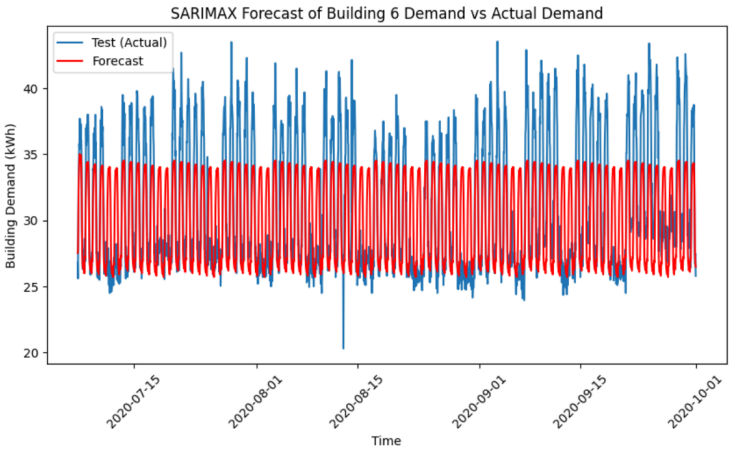

Figure 21 - Forecast of Building 6 on testing data using ARIMA model

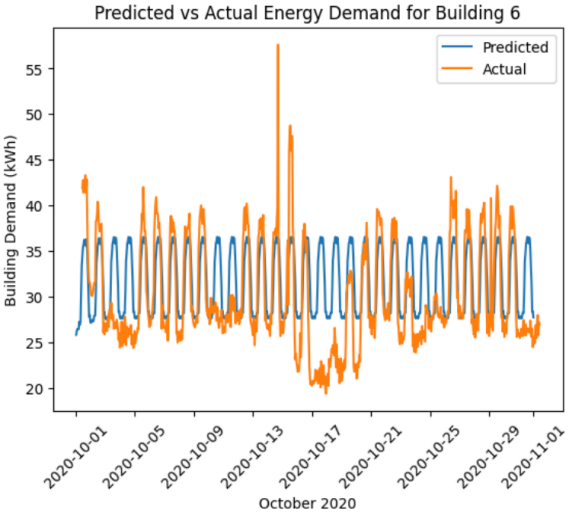

Figure 22 - Comparison of Building 6 October predictions to actual October values using ARIMA

Solar Forecasting

For the solar dataset, p, q, and d values of 1 were used to forecast Solar 1 output for October 2020. Stationarity was assessed using the Augmented Dickey–Fuller (ADF) test (Yenkikar et al., 2025), which produced a statistic of -0.588 and a p-value of 0.874, indicating that the series retained a strong trend component and was non-stationary.

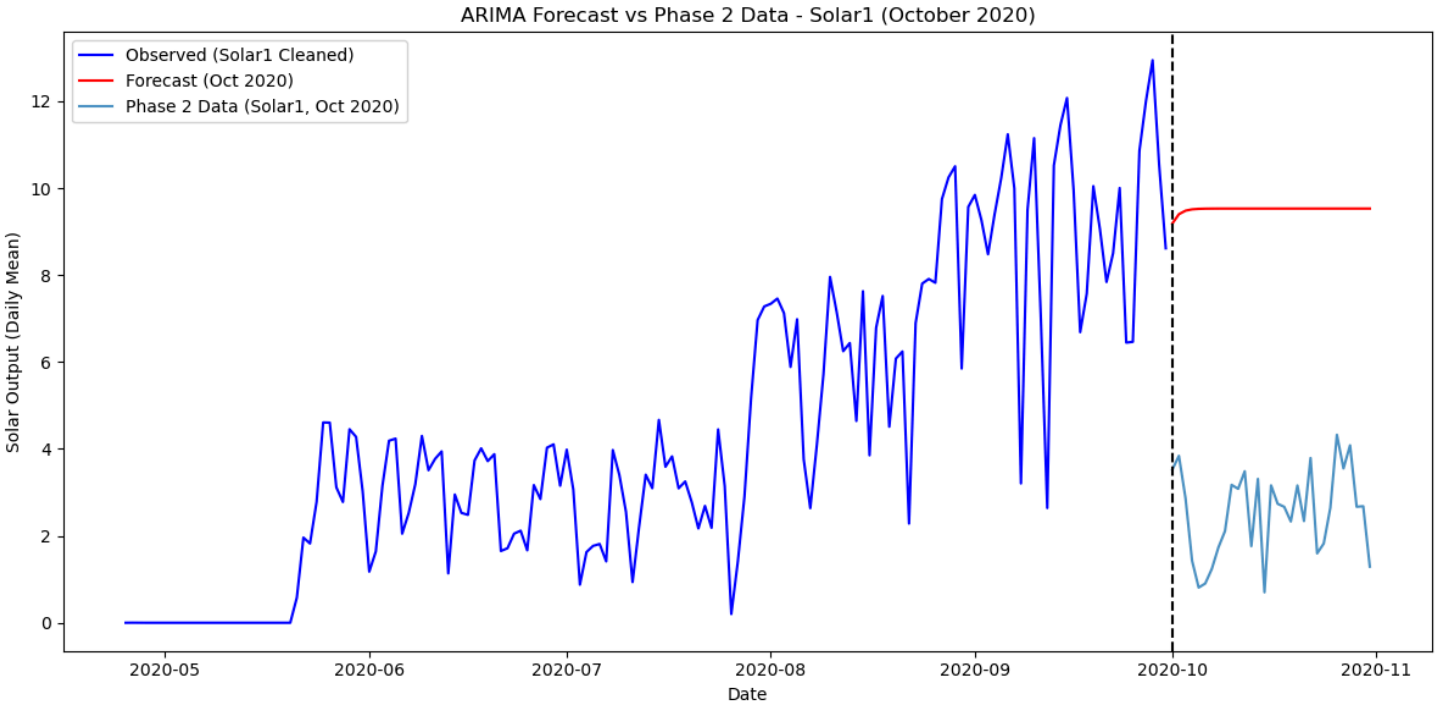

Figure 23 - Forecast of Solar 1 for October 2020 using ARIMA

As seen in **Figure 23**, the forecast consistently overestimated output by almost threefold and failed to capture daily trends. This occurred because the model generalised historical patterns, expecting conditions similar to the training period; for example, a sunny September led the model to predict high values for October, even when weather was poorer. ARIMA's reliance solely on past values also meant it could not reproduce day-to-day fluctuations.

The error metrics further highlight the model's poor performance: MAE was 3.89, showing average deviations of nearly four units, and RMSE was 4.55, indicating larger discrepancies on some days - considerably high given that values rarely exceed 12 kWh. The strong correlations among the six solar panels (mostly above 0.9) observed in the EDA suggest that generation is heavily influenced by common factors such as sunlight, reinforcing the importance of including weather features. Incorporating SARIMA or SARIMAX, as done for the buildings, could improve accuracy by accounting for both seasonality and exogenous weather variables.

Scheduling Optimisation (Gurobi)

Parsing Instance Files

To begin the optimisation problem the instance files needed to be broken up so its data could be readable. Therefore the instance file parser was created to extract the structured information that was given in the raw txt instance files. The parser begins by validating the file header to confirm that it is the expected "ppoi" (predict-plus-optimise instance) format, which includes the number of buildings, solar arrays, batteries, recurring activities and once-off activities. The parser then iterates through each remaining line and classifies it based on its prefix (b,s,c,r,a) extracting the relevant numerical and categorical data accordingly. Building entries (starting with "b") store number of small and large rooms, solar lines define which solar arrays are linked to which buildings, battery entries store data on their capacity, power and efficiency.

For activity entries, the parser separates recurring and once-off activities, capturing the number of rooms they require, room sizes, power load for each room and duration. Recurring activity lines also contain the activity IDs of any precedence activities that need to be scheduled before them during the week, whilst once-off activity entries contain data on the dollar amount gained from scheduling them and the dollar amount penalty that comes with scheduling them outside of office hours. This information is stored in nested python dictionaries and lists, which allows the optimisation model to easily access, index and manipulate the data.

Model Version 1

The first version of the Gurobi optimisation model was developed as a foundational proof of concept to ensure that the overall scheduling and optimisation structure is robust before actually integrating real energy and solar data. In this version, several assumptions were made to reduce complexity and ensure that the model executed successfully. Specifically, the base building net loads were assumed to be zero, meaning that each building was treated as having no background energy demand aside from the scheduled activities. Furthermore, dynamic energy pricing was not yet implemented but instead a constant cost value was used in the objective function. The optimisation objective itself was also different from the final specification, V1 aimed to minimise the peak grid draw P_{peak}, rather than minimising total energy cost over the month. This meant that the model focused solely on flattening the load profile rather than considering variations in energy prices, building net loads, solar production data and scheduling in the most cost effective way.

Battery modelling was included at this stage but in a simplified form. Batteries charged exclusively from the grid, rather than from both solar and grid sources. The state-of-charge (SOC) dynamics were still correctly constrained using charging and discharging efficiencies, ensuring that SOC evolved consistently through time. These simplifications allowed the model to focus on having correct structure, verifying that time-indexed constraints, activity scheduling and precedence relationships were properly integrated.

During execution, the model created binary decision variables for all possible recurring and once-off activity start times and continuous variables for battery charging, discharging, and SOC. Constraints were added to ensure each recurring activity was scheduled exactly once per week and that all once-off activities could only start once in the month. A linking constraint tied total grid power draw at each timestep P_t to the variable P_{peak}, enforcing $P_t \leq P_{peak}$ across all time intervals. The objective function then minimised P_{peak}, leading the solver to reduce the highest instantaneous power demand across its planning.

The solution output was structured into three main sections:

1. Peak grid draw summary, reporting the final P_{peak} value.

2. Scheduled recurring and once-off activities, showing tuples with activity IDs, week/day/time indices, and their corresponding global start steps (as seen in **Appendix B & C**).

3. Battery actions, showing the charging and discharging behaviour per building battery (as seen in **Appendix D**).

In this version, the reported value of P_{peak} was zero with a gap of 0%, which indicates that the model found a mathematically feasible solution where total grid draw was set to zero. This outcome was a direct consequence of the simplifications used, specifically, the net building loads being zero and no penalty or reward being applied to activity power usage. As a result, the optimisation did not need to generate any non-zero grid power flow to satisfy the objective, therefore producing an ineffective solution.

Overall, Version 1 successfully demonstrated that the model architecture, decision variable definitions and constraint logic were functional. However, it also highlighted the need for more realistic system dynamics. V2 therefore incorporated real building load data, time varying electricity prices, integration of solar production data and a revised objective function to minimise total energy cost.

Model Version 2 - Final Version

The final optimisation model builds upon Version 1 by utilising more realistic data and a fully cost driven objective. Four aligned input datasets were used. First was the energy price data at 15-minute intervals derived from the Australian Energy Market Operator (AEMO) for November 2020 (see **Appendix E**). Second was the predicted building energy consumption across all 6 buildings(see **Appendix F**). Third was the solar energy generation forecasts for each rooftop array (see **Appendix G**) and lastly the phase 2 (November 2020) instance file itself describing building capacities, batteries, and activities. These were synchronised into a unified 15-minute index spanning four weeks for the month (96 steps × 28 days = 2688 steps).

The model was formulated as a mixed-integer linear program (MILP) and solved using Gurobi.

Binary variables were introduced for:

$x_{a,w,d,s}$ recurring activity *a* starting in week *w*, day *d*, step *s*;
$y_{a,s}$ once-off activity *a* starting globally at step *s*;
z_a a binary indicator indicating whether once-off activity*a*is scheduled at all.

Continuous variables represented battery and solar-energy flows:

$p_{b,t}^{charge}$ battery *b* charging power from grid (kW)
$p_{b,t}^{chargePV}$ battery *b* charging power from solar (kW)
$p_{b,t}^{discharge}$ battery *b*discharging on load (kW)
$s_{b,t}$ state of charge (kWh)
$P_{b,t}^{PV \to Load}$, $P_{b,t}^{PV \to Battery}$, $P_{b,t}^{PV \to Reduced}$ solar energy routed to load, battery or reduced

Each variable was defined for every building and time step $t \in [0, T-1]$.

Key Constraints

1. Activity scheduling

Every recurring activity must start exactly once per week: $\sum_{(d,s)\in\Omega_a}^{n} x_{a,w,d,s} = 1 \; \forall a, w$

and each once-off activity may start at most once: $\sum_{s}^{n} y_{a,s} = z_a \; \forall a$

2. Battery state of charge (SOC) dynamics

The SOC evolution is continuous over time, integrating charge and discharge flows while respecting round-trip efficiency η:

$$s_{b,t} = s_{b,t-1} + \left(\left(P_{b,t}^{charge} + P_{b,t}^{chargePV}\right)\sqrt{\eta} - \frac{P_{b,t}^{discharge}}{\sqrt{\eta}}\right)\Delta t$$

With $s_{b,0} = C_b$ (initially full) and $0 \leq s_{b,t} \leq C_b$

3. Solar-power allocation

At each step and building, total solar energy generation was distributed among load supply, battery charging, and reduction:

$P_{b,t}^{PV \text{ -> } Load}$, $P_{b,t}^{PV \text{ -> } Battery}$, $P_{b,t}^{PV \text{ -> } Reduced} \leq P_{b,t}^{PV}$

4. Grid-draw calculation and non-negativity

For every 15-minute interval, total grid power draw was computed as:

$$p_t = \sum_{b}\left(P_{b,t}^{base} + P_{b,t}^{activity} - P_{b,t}^{PV \to load}\right) + \sum_{b}\left(\frac{P_{b,t}^{charge}}{\sqrt{\eta}} - P_{b,t}^{discharge}\sqrt{\eta}\right)$$

with the constraint $p_t \geq 0$ and peak-tracking variable $P_{peak} \geq p_t$

5. Precedence relationships

For both recurring and once-off activities with dependencies, start-time linear constraints were added to ensure that each predecessor completed before its successor:

$s_j + 1 \leq s_k$

Objective Function

The model maximised **net profit**, expressed as:

$$\max Z = \sum_{a,s} y_{a,s}\left(v_a - \pi_a * I_{a,s}^{out}\right) - \sum_{t} p_t * \rho_t * \Delta t$$

Where

v_a is the base value of once-off

a, π_a is the penalty for starting outside working hours, $I_{a,s}^{out}$ is a binary flag indicating such cases, p_t is the energy price ($/kWh) and $\Delta t = 0.25$ hours.

Solver Configuration

The MILP was solved using Gurobi 10.0 with an upper time limit of 1.5 hours (5400 seconds) to ensure tractable runtime. The solver achieved a relative optimality gap of 2.5783%, meaning that the best feasible solution found ($Z_{best} = 5.2847 * 10^6$) was within 2.6 % of the theoretical best bound ($Z_{bound} = 5.4209 * 10^6$). The *MIP gap* quantifies the remaining distance between the best found solution and the best known possible value, so while the model did not reach mathematical optimality before the time cap, the solution is near-optimal and operationally sound.

Results

The final schedule achieved a net profit of approximately -$5.28 million for the month, with a peak grid draw of 37 MW. The optimiser determined optimal recurring activity placements and once-off scheduling (see **Appendix X and X**) that balanced daily load profiles against fluctuating electricity prices. Batteries were charged dynamically from both grid and solar sources, utilising solar energy preferentially during daylight hours (**Appendix X).**
The high monthly profit reflects the magnitude of total energy flows predicted in the building and solar datasets. Variability and potential over-estimation in the underlying forecasts can magnify outcomes, particularly since energy cost is a product of both demand (kWh) and price ($/kWh). Future improvements could therefore include refining forecast accuracy, incorporating real measured solar and load data, and adjusting model parameters such as battery efficiencies or demand aggregation to produce more realistic financial outcomes.

Overall, Version 2 successfully integrates time varying prices, battery and solar coupling, and detailed operational constraints to produce an economically grounded, near-optimal monthly energy schedule for the Monash Solar Farm system.

	Activity ID	Activity	Week	Day	Start (hh:mm)	End (hh:mm)	Global Start	Duration (steps)	Duration (min)	Rooms (#, Size)	Load per room (kW)	Total Load (kW)
72	0	Recurring 0	1	Mon	1:15 PM	3:15 PM	53	9	135	2 S	70.0	140.0
202	0	Recurring 0	2	Mon	11:15 AM	1:15 PM	717	9	135	2 S	70.0	140.0
789	0	Recurring 0	4	Tue	9:45 AM	11:45 AM	2151	9	135	2 S	70.0	140.0
405	0	Recurring 0	3	Tue	11:45 AM	1:45 PM	1487	9	135	2 S	70.0	140.0
493	1	Recurring 1	3	Tue	12:45 PM	2:45 PM	1491	9	135	2 S	100.0	200.0
...	...	...	...	...	...	...	...	...	...	...	...	...
292	198	Recurring 198	2	Mon	12:00 PM	12:30 PM	720	3	45	2 S	88.0	176.0
492	199	Recurring 199	3	Tue	12:30 PM	1:45 PM	1490	6	90	1 S	65.0	65.0
293	199	Recurring 199	2	Mon	12:00 PM	1:15 PM	720	6	90	1 S	65.0	65.0
126	199	Recurring 199	1	Thu	2:00 PM	3:15 PM	344	6	90	1 S	65.0	65.0
712	199	Recurring 199	4	Tue	11:30 AM	12:45 PM	2158	6	90	1 S	65.0	65.0

Figure 24 - Optimal V2 Recurring Activity Schedule

Conclusion & Improvements

The final Gurobi optimisation model (Version 2) showed a significant improvement in both cost efficiency and operational performance compared to the baseline instance file schedule. By incorporating dynamic energy pricing, real building net loads, and solar generation forecasts, the optimiser achieved a total energy cost of $5,284,701, representing a 13.49% reduction from the instance file's cost of $6,107,313. Additionally, the model successfully reduced the peak grid draw from 75,916 kW to 37,692 kW, almost a 50% decrease, resulting in a smoother and more energy-efficient load profile. Revenue from once-off activities also improved markedly, increasing from $4,660 to $8,860, as the optimiser strategically scheduled all available once-off events (as seen in **Appendix H**)

To have a fair comparison between the two schedules, an evaluator function was developed that recalculated energy costs for the baseline instance using the same building load and solar production data as Version 2. This ensured both models were assessed under identical operational conditions, making the comparison relative and meaningful. Notably, Version 2 scheduled all once-off activities, including those outside standard office hours, as it determined that extending activity scheduling outside working hours and getting penalised maximised overall revenue. Future improvements could therefore include adding time-of-day preference constraints or incorporating labour-hour penalties to achieve a more practical balance between cost and feasibility.

Another minor limitation identified was within the recurring activity precedence constraints, where the model allowed a successor activity to begin one hour after its predecessor, even if the predecessor's duration extended beyond that time. Although this caused minimal overlap and had a negligible effect on total outcomes, refining the constraint to prevent such overlaps would improve accuracy. However, due to time constraints and the increase it would have on computation time, the issue was not corrected in this version.

Conclusion

This project set out to predict the electricity demand and solar power production from six buildings at Monash University based on predictions of the weather during this time, and subsequently use these forecasts to optimise the timetable and solar battery usage.

Through our experimentation, modelling and optimisation we found that ultimately LightGBM outperformed ARIMA for solar production, achieving lower MAE and RMSE values than ARIMA. Conversely, ARIMA was more accurate for individual building electricity demands, with a lower MAPE of 13.86% compared to LightGBM's 59.07%. However, ARIMA could not be fully scaled across all buildings due to memory limitations.

Using these forecasts in a Gurobi optimisation model successfully minimised overall energy costs and peak grid demand. The optimised timetable achieved a 13.49% cost improvement over the instance files provided, with the optimised timetable reducing total energy expenditure from $6,107,313 to $5,283,815, and cutting the peak grid draw to 49.6% of the original level. These results demonstrate that accurate forecasting combined with constrained optimisation substantially improves renewable energy scheduling efficiency.

However, despite these gains, several limitations affected the model's accuracy, mainly the difficulty of accurately forecasting weather for a month in advance without access to all the required feature variables. The limited number of weather stations and data being collected at the relevant weather stations reduced the predictive strength of the weather forecasting. Additionally, the use of a global LightGBM model without fine-tuning for individual buildings led to higher error measures for the building electricity demand forecasts. The ARIMA model may have also been a suitable choice for predicting building demand if there were no memory errors.

Whilst this project delivered large cost reductions, future work could address these issues by integrating additional external weather datasets, develop per-building models, and applying neural networks to capture long-term temporal patterns. These improvements could enhance forecasting accuracy, and, in turn, enable further optimisation of the timetable and energy costs.

References

Bashab, A., Osman Ibrahim, A., Abakar Tarigo Hashem, I., Aggarwal, K., Mukhlif, F., A. Ghaleb, F., & Abdelmaboud, A. (2023). Optimization Techniques in University Timetabling Problem: Constraints, Methodologies, Benchmarks, and Open Issues. *Computers, Materials & Continua, 74*(3), 6461–6484. https://doi.org/10.32604/cmc.2023.034051

Didwania, G. (2025, July 17). *Mastering Time Series Forecasting with LightGBM: A Practical Guide.* Medium; Data Science Collective. https://medium.com/data-science-collective/mastering-time-series-forecasting-with-lightgbm-a-practical-guide-2dff8d1a72bb

IEEE Xplore Full-Text PDF. (2025). *Ieee.org.* https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=8653826

Junninen, H., Niska, H., Tuppurainen, K., Ruuskanen, J., & Kolehmainen, M. (2004). Methods for imputation of missing values in air quality data sets. *Atmospheric Environment (1994), 38*(18), 2895–2907. https://doi.org/10.1016/j.atmosenv.2004.02.026

Kontopoulou, V. I., Panagopoulos, A. D., Kakkos, I., & Matsopoulos, G. K. (2023). A Review of ARIMA vs. Machine Learning Approaches for Time Series Forecasting in Data Driven Networks. *Future Internet, 15*(8), 255. https://doi.org/10.3390/fi15080255

Kuppannagari, S. R., Fu, Y., Chueng, C. M., & Prasanna, V. K. (2021). Spatio-Temporal Missing Data Imputation for Smart Power Grids. *Proceedings of the Twelfth ACM International Conference on Future Energy Systems*, 458–465. https://doi.org/10.1145/3447555.3466586

Medar, R., Angadi, A. B., Niranjan, P. Y., & Tamase, P. (2017). Comparative study of different weather forecasting models. 1604–1609. https://doi.org/10.1109/icecds.2017.8389719

Prakash, C., Dhyani, B., Chauhan, A., & Sayal, A. (2025). ARIMA based forecasting of solar and hydro energy consumption with implications for grid stability and renewable policy. *Discover Sustainability, 6*(1), Article 663. https://doi.org/10.1007/s43621-025-01536-8

Ugbehe, P. O., Diemuodeke, O. E., & Aikhuele, D. O. (2025). Electricity demand forecasting methodologies and applications: a review. *Renewables: Wind, Water, and Solar, 12*(1), Article 19. https://doi.org/10.1186/s40807-025-00149-z

Wan, H., Wang, J., Gan, Q., Xia, Y., Chang, Y., & Yan, H. (2025). Addressing intermittency in medium-term photovoltaic and wind power forecasting using a hybrid xLSTM-TCCNN model with numerical weather predictions. *Renewable Energy, 253,* 123618. https://doi.org/10.1016/j.renene.2025.123618

Yenkikar, A., Mishra, V. P., Bali, M., & Ara, T. (2025). Explainable forecasting of air quality index using a hybrid random forest and ARIMA model. *MethodsX, 15,* Article 103517. https://doi.org/10.1016/j.mex.2025.103517

Zhao, G., Yu, C., Huang, H., Yu, Y., Zou, L., & Mo, L. (2025). Optimization Scheduling of Hydro–Wind–Solar Multi-Energy Complementary Systems Based on an Improved Enterprise Development Algorithm. *Sustainability*, *17*(6), 2691. https://doi.org/10.3390/su17062691

Acknowledgments

We would like to thank Simon Clarke, Russell Tsuchida, and Jeffery Liu for their individual and group feedback on the methodology and data cleaning.

Appendix

Appendix A - Time Classification Function

```python
def classify_time(hour):
    if 6 <= hour < 12:
        return 'Morning'
    elif 12 <= hour < 16:
        return 'Afternoon'
    elif 16 <= hour < 22:
        return 'Evening'
    else:
        return 'Night'

df_all['time_period'] = df_all['timestamp'].dt.hour.map(classify_time)
```

The following Python function was used to classify each timestamp into time-of-day categories for exploratory data analysis.

Appendix B - Optimised Recurring Activity Schedule

```
(0, 0, 0, 36, 36)
(0, 1, 0, 36, 708)
(0, 2, 0, 36, 1380)
(0, 3, 0, 36, 2052)
```

Appendix C - Optimised Once-off Activity Schedule

```
(0, 2259)
(1, 1397)
```

Appendix D - Optimised Battery Schedule

```
Sample battery actions:
Battery 0 sample events: [(37, 'charge', 1.0172375127149682), (38, 'charge
', 1.9239156025945505), (39, 'charge', 2.2417456938838614), (40, 'charge',
1.1570581148906915), (41, 'charge', 2.021235669307169), (42, 'charge', 1.6
008499380960173), (43, 'charge', 0.7403680824154621), (44, 'charge', 1.217
1132193494276), (45, 'charge', 0.6445878901761452), (46, 'charge', 0.22851
024314261478)]
Battery 1 sample events: [(36, 'charge', 5.671192618647703), (37, 'charge'
, 7.0), (38, 'charge', 5.450806661517032), (39, 'charge', 8.54919333848296
7), (40, 'charge', 5.450806661517032), (41, 'charge', 7.0), (42, 'charge',
7.0), (43, 'charge', 8.549193338482967), (44, 'charge', 5.450806661517032)
, (45, 'charge', 7.0)]
```

Appendix E - Energy Price Data (AEMO) Imputed

	REGION	SETTLEMENTDATE	TOTALDEMAND	RRP	PERIODTYPE
0	VIC1	2020/11/01 00:30:00	4252.61	75.16	TRADE
1	VIC1	2020/11/01 01:00:00	4119.61	69.14	TRADE
2	VIC1	2020/11/01 01:30:00	4015.35	52.61	TRADE
3	VIC1	2020/11/01 02:00:00	3844.49	52.02	TRADE
4	VIC1	2020/11/01 02:30:00	3759.76	48.85	TRADE
...	...	...	...	...	...
1435	VIC1	2020/11/30 22:00:00	4816.41	54.50	TRADE
1436	VIC1	2020/11/30 22:30:00	4640.49	59.37	TRADE

Appendix F - Building Net Load Predictions

	timestamp	B0	B1	B2	B3	B4	B5
0	2020-11-01 00:00:00	216.735419	22.630197	715.467571	1.266667	26.765690	34.579138
1	2020-11-01 00:15:00	208.327834	5.844461	737.479363	1.317647	27.579167	34.616780
2	2020-11-01 00:30:00	208.327834	11.704704	737.500295	1.317518	27.850622	34.618594
3	2020-11-01 00:45:00	208.327834	17.527921	737.500295	1.348837	26.203320	34.477828
4	2020-11-01 01:00:00	208.323389	23.333080	737.500295	1.314394	27.724280	34.671946
...	...	...	...	...	...	...	...
2875	2020-11-30 22:45:00	226.342594	16.249088	668.344320	1.334520	26.983264	33.453968
2876	2020-11-30 23:00:00	226.342594	21.639058	668.344320	1.345070	27.520833	34.032653
2877	2020-11-30 23:15:00	216.747610	5.569196	715.501473	1.351254	27.228216	34.102494
2878	2020-11-30 23:30:00	216.747610	11.253263	715.386792	1.363636	26.644628	34.085714
2879	2020-11-30 23:45:00	216.747610	16.953263	715.399646	1.363309	27.629167	34.008617

Appendix G - Solar Production Predictions

	timestamp	S0	S1	S2	S3	S4	S5
0	2020-11-01 00:00:00	0.010333	0.0	0.0	0.0	0.0	0.0
1	2020-11-01 00:15:00	0.010333	0.0	0.0	0.0	0.0	0.0
2	2020-11-01 00:30:00	0.010333	0.0	0.0	0.0	0.0	0.0
3	2020-11-01 00:45:00	0.010333	0.0	0.0	0.0	0.0	0.0
4	2020-11-01 01:00:00	0.010333	0.0	0.0	0.0	0.0	0.0
...	...	...	...	...	...	...	...
2875	2020-11-30 22:45:00	0.010333	0.0	0.0	0.0	0.0	0.0
2876	2020-11-30 23:00:00	0.010333	0.0	0.0	0.0	0.0	0.0

Appendix H - Optimal V2 Once-off Activity Schedule

	Activity ID	Activity	Week	Day	Start (hh:mm)	End (hh:mm)	Global Start	Duration (steps)	Duration (min)	Outside Work Hours?	Value ($)	Penalty Applied ($)	Realized Value ($)	Rooms (#, Size)	Load per room (kW)	Total Load (kW)
0	84	OnceOff 84	1	Sun	4:15 AM	5:45 AM	593	7	105	True	17.0	17.0	0.0	3 S	77.0	231.0
1	16	OnceOff 16	1	Sun	5:00 AM	5:45 AM	596	4	60	True	15.0	8.0	7.0	3 S	77.0	231.0
2	3	OnceOff 3	1	Sun	5:15 AM	6:00 AM	597	4	60	True	19.0	10.0	9.0	3 S	102.0	306.0
3	28	OnceOff 28	1	Sun	5:15 AM	6:00 AM	597	4	60	True	12.0	6.0	6.0	2 L	96.0	192.0
4	45	OnceOff 45	1	Sun	5:15 AM	5:45 AM	597	3	45	True	3.0	1.0	2.0	1 S	63.0	63.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
95	23	OnceOff 23	3	Tue	1:45 PM	2:00 PM	1495	2	30	False	5.0	0.0	5.0	2 S	97.0	194.0
96	14	OnceOff 14	4	Mon	5:30 AM	7:00 AM	2038	7	105	True	16.0	7.0	9.0	2 S	95.0	190.0
97	30	OnceOff 30	4	Mon	6:00 AM	7:00 AM	2040	5	75	True	20.0	13.0	7.0	3 S	86.0	258.0
98	37	OnceOff 37	4	Mon	6:00 AM	7:15 AM	2040	6	90	True	9.0	8.0	1.0	2 S	66.0	132.0
99	94	OnceOff 94	4	Mon	6:00 AM	7:00 AM	2040	5	75	True	4.0	2.0	2.0	1 S	61.0	61.0